

CYBER RANGE

DoS Lab

Technical Operations & Command Reference Manual

Document Type	Technical Reference Manual
Classification	Confidential — Internal Use Only
Audience	Security Engineers, Analysts & Researchers
Version	1.0

Authors	
Marwan Abdelaziz Mohamed	231017947
Mostafa Mohamed Khafaga	231027979
Ziad Khaled Youssef	231017416

1. Executive Summary

This document serves as the comprehensive technical operations and command reference manual for the Cyber Range DoS Lab project. It is designed to provide full transparency into every execution stage, script, terminal operation, and backend command utilized across the project's lifecycle.

By detailing the precise parameters, internal mechanics, expected outputs, and troubleshooting procedures of each command, this manual empowers engineers, security analysts, and researchers to:

- Rebuild the environment from scratch with confidence
- Execute every component correctly and in the correct sequence
- Deeply understand the internal workflow of the entire federated system

2. Project Organization & Initialization

Before any container is launched, the project directory must be systematically organized. This is handled by the initial setup script.

2.1 File Organization — `organize.ps1`

Purpose: Bootstraps the physical directory structure and stages all configuration files into their appropriate service-specific folders.

Run the script with the following command:

```
./organize.ps1
```

Internal operations performed by this script:

- Directory Creation (New-Item -ItemType Directory -Force): Systematically builds the workspace hierarchy, including victim/website, attacker/scripts, packet_capture, and monitoring/grafana/dashboards.
- File Migration (Move-Item -Force): Migrates root-level .sh, .yaml, .conf, and .html files into their respective service directories.
- Dynamic Dockerfile Generation (Set-Content): Injects hardcoded Dockerfile configurations for the Victim (Apache), Attacker (hping3, siege, vegeta), and Packet Capture (tshark) containers directly into the file system.

Expected Outcome: A perfectly structured project directory ready for Docker Compose orchestration.

3. Infrastructure Deployment & Automation

The project utilizes Docker Compose to orchestrate a multi-network environment spanning Attacker, Victim, Monitoring, and Logging networks. Deployment is managed via Bash and PowerShell wrapper scripts.

3.1 Automated Execution Flow — run_auto.ps1

Purpose: Provides a one-click autonomous execution of the entire lab lifecycle, including deployment, simulation, visualization, and teardown.

```
./run_auto.ps1
```

The automation script executes the following steps in sequence:

Step 1 — Infrastructure Build

```
docker compose --profile attack --profile mitigation up -d --build
```

Deploys the infrastructure in detached mode (-d) and forces an image rebuild (--build). The --profile flags ensure optional services (attacker node and mitigation reverse proxy) are included in the deployment.

Step 2 — Health Verification

```
docker inspect --format '{{.State.Health.Status}}' dos_victim
```

Loops and queries the Docker daemon for the health status of the victim container. Script execution halts until this returns healthy.

Step 3 — Background Packet Capture

```
docker exec -d dos_packet_capture bash /entrypoint.sh
```

Executes the packet capture routine inside the isolated dos_packet_capture container in detached mode (-d). This ensures traffic is recorded without blocking the automated script's execution.

Step 4 — Dashboard Launch

```
Start-Process "http://localhost:3000"
```

Automatically opens the default system web browser to the Grafana metrics dashboard.

Step 5 — Attack Execution

```
docker exec dos_attacker bash /scripts/simulate_load.sh
```

Synchronously triggers the load simulation script on the attacker container. The terminal awaits completion of the simulated DoS attack.

Step 6 — Graceful Packet Capture Termination

```
docker exec dos_packet_capture pkill -SIGINT tshark
```

Sends a SIGINT (Interrupt) signal to the tshark process inside the capture container. This ensures the PCAP file is safely flushed to disk and closed without corruption.

4. Attack Execution & Load Simulation

The attacker container (dos_attacker) houses a suite of network stress-testing tools. Orchestration is handled by simulate_load.sh and attack.sh.

4.1 Load Simulation Trigger — attack.sh

Provides a centralized entry point to launch specific DoS methodologies from the host machine into the attacker container.

```
./scripts/attack.sh [SCENARIO]
```

Available scenarios:

Command / Parameter	Description
ab	Apache Benchmark HTTP flood
vegeta	Constant-rate load testing
siege	Concurrent multi-URL load simulation
http_load	Asynchronous Python HTTP load
hping3	Network-layer packet injection
all	Execute all scenarios sequentially

4.2 Traffic Generation Commands — simulate_load.sh

4.2.1 Apache Benchmark (HTTP Flood)

```
ab -n 1000 -c 10 -t 30 -k -H "Accept-Encoding: gzip, deflate" -r  
http://172.21.0.10/
```

Command / Parameter	Description
-n 1000	Maximum number of requests to perform
-c 10	Number of concurrent requests
-t 30	Maximum seconds to spend benchmarking
-k	Enable HTTP KeepAlive feature
-r	Do not exit on socket receive errors

Behavior: Generates a rapid burst of HTTP GET requests designed to exhaust Apache worker threads.

4.2.2 Vegeta (Constant-Rate Load)

```
echo "GET http://172.21.0.10/" | vegeta attack -rate=100 -duration=30s | vegeta  
report
```

Command / Parameter	Description
<code>vegeta attack</code>	Reads target URLs from standard input
<code>-rate=100</code>	Enforces a strict rate of 100 requests per second
<code>-duration=30s</code>	Sustains the attack for 30 seconds
<code> vegeta report</code>	Pipes binary output to a human-readable statistical report

Behavior: Provides extremely precise rate-based HTTP loading, ideal for testing threshold-based rate limiting.

4.2.3 Siege (Concurrent Multi-URL Load)

```
siege --concurrent=10 --time=30S --file=/tmp/siege_urls.txt
```

Simulates multiple users accessing diverse endpoints simultaneously, mimicking complex user behavior rather than a simple single-endpoint flood.

4.2.4 Hping3 (Network Layer Packet Injection)

```
# ICMP Ping mode
hping3 -c 10 -l --fast 172.21.0.10

# TCP SYN Probe
hping3 -c 5 -S -p 80 172.21.0.10
```

Command / Parameter	Description
<code>-c 10</code>	Count: send exactly 10 packets
<code>-l</code>	ICMP mode
<code>-S</code>	Set the SYN TCP flag (simulates TCP handshake initiation)
<code>-p 80</code>	Target destination port 80

Behavior: Used for educational demonstration of raw packet crafting at OSI Layers 3 and 4.

4.2.5 Asynchronous Python HTTP Load — `http_load.py`

```
python3 /scripts/http_load.py --target 172.21.0.10 --rate 50 --concurrency 10
```

Utilizes Python's `asyncio` and `aiohttp` libraries to generate highly concurrent, non-blocking asynchronous HTTP requests. Automatically calculates p95/p99 latencies and standard deviations.

5. Packet Capture & Digital Forensics (DFIR)

The `dos_packet_capture` container acts as an out-of-band network tap spanning the lab networks, enabling passive recording and forensic analysis of all traffic.

5.1 Tshark Packet Capture

```
tshark -i eth0 -f "tcp port 80" -b duration:60 -w /captures/traffic.pcap
```

Command / Parameter	Description
<code>-i eth0</code>	Listen on the primary network interface
<code>-f "tcp port 80"</code>	BPF filter capturing only HTTP traffic
<code>-b duration:60</code>	Ring-buffer rotation: new file every 60 seconds
<code>-w /captures/...</code>	Write binary output to the specified PCAP file

5.2 DFIR Analysis Script — `analyze_pcap.sh`

Executes automated forensic queries against captured PCAP files. The following commands are used in sequence during analysis:

General Capture Statistics

```
tshark -r traffic.pcap -q -z io,stat,0
```

Generates a high-level summary of packet rates, byte counts, and capture duration.

Protocol Hierarchy Statistics

```
tshark -r traffic.pcap -q -z io,phs
```

Displays a tree view of all protocols present in the capture (Ethernet > IPv4 > TCP > HTTP).

Top Source IP Distribution

```
tshark -r traffic.pcap -T fields -e ip.src | sort | uniq -c | sort -rn | head -20
```

Extracts source IP fields, counts occurrences, and sorts to identify top bandwidth consumers and confirm attacker IP addresses.

SYN Flood Identification

```
tshark -r traffic.pcap -Y "tcp.flags.syn==1 && tcp.flags.ack==0" -T fields -e frame.number | wc -l
```

Filters for packets containing only a SYN flag. High ratios of SYN vs. SYN-ACK indicate a potential TCP state exhaustion attack.

HTTP Method Distribution

```
tshark -r traffic.pcap -Y "http.request" -T fields -e http.request.method | sort | uniq -c
```

Extracts HTTP methods (GET, POST) to verify the nature and composition of the application-layer attack.

6. Environment Export & Cleanup

Proper lab hygiene is maintained through robust export and teardown scripts. These ensure all artifacts are preserved before infrastructure destruction.

6.1 Exporting Reports — export_reports.sh

```
./scripts/export_reports.sh
```

Log Extraction

```
docker exec dos_victim bash -c "cat /var/log/apache2/access.log" > apache_logs/access.log
```

Pulls internal container logs into the host filesystem prior to container destruction.

Archiving

```
tar -czf lab_export_TIMESTAMP.tar.gz -C ./exports lab_export_TIMESTAMP
```

Bundles PCAPs, generated analysis markdown, and raw logs into a compressed, portable tarball for long-term retention.

6.2 Full Environment Teardown — cleanup.sh

```
./scripts/cleanup.sh
```

Container & Volume Destruction

```
docker compose down -v --remove-orphans
```

Stops all compose-managed containers, removes all attached networks, and deletes all anonymous and named volumes (-v). This resets Prometheus metrics, Grafana configurations, and Loki log states.

File Deletion

```
rm -f ./logs/*  
rm -f ./packet_capture/captures/*.pcap
```

Wipes temporary runtime files while strictly preserving all generated final reports.

7. Troubleshooting & Diagnostics

If the lab environment fails to initialize or does not exhibit expected behavior, consult the following diagnostic procedures:

Issue	Diagnostic Command	Recommended Fix
Containers failing to start / port conflicts	<code>docker ps -a</code>	Ensure ports 8080, 3000, 9090, 8081, 3100, and 8443 are free. Run <code>./scripts/stop_lab.sh</code> to clear zombie containers.
Attack scripts error: Container Not Found	<code>docker compose ps</code>	Boot the attacker container using the correct compose profile: <code>docker compose --profile attack up -d</code>
No PCAP data generated	<code>docker logs dos_packet_capture</code>	Verify the target interface is correctly mapped and the container has <code>cap_add: [NET_ADMIN, NET_RAW]</code> privileges in <code>docker-compose.yml</code> .
Grafana dashboards show 'No Data'	<code>docker inspect dos_prometheus</code>	Navigate to <code>http://localhost:9090/targets</code> to verify Prometheus scraping. Check network bridges if targets appear as DOWN.

8. Operating System Compatibility

8.1 Windows Hosts

- Run scripts via PowerShell (`run_auto.ps1`, `organize.ps1`).
- Ensure Docker Desktop is configured for the WSL2 backend.
- All `.sh` scripts must use LF line endings (not CRLF) to execute correctly inside Linux containers.

8.2 Linux & macOS Hosts

- Native execution supported via: `bash ./scripts/start_lab.sh`
- Execute `chmod +x` on all scripts within `/scripts` and `/attacker/scripts` prior to runtime.

END OF DOCUMENT — CONFIDENTIAL